

Now, suppose that we want to delete the record that has the field title equal to A view of Data types in C++. The table posts has the pk field as the primary key, and for the record A view of Data types in C++, the value of pk is equal to 7; so first of all, let's delete the records from the j_posts_tags table for which the value post_pk=7. This is because there is a foreign key that links the posts and j_posts_tags tables:

```
forumdb=> delete from j_posts_tags where post_pk = 7;
DELETE 1
```

Now let's call the delete_posts function using A view of Data types in C++ as the parameter. This is the situation after we called the delete_posts function:

```
forumdb=> select delete_posts('A view of Data types in C++');
delete_posts
-----
              7
(1 row)
forumdb=> select pk,title from posts order by pk;
 pk |      title
----+-----
  5 | Indexing PostgreSQL
  6 | Indexing Mysql
(2 rows)
```

In this function, we've introduced a new kind of data type – the setof data type. The setof directive simply defines a result set of a data type. For example, the delete_posts function is defined to return a set of integers, so its result will be an integer dataset. We can use the setof directive with any type of data.

SQL functions returning a table

In the previous section, we saw how to write a function that returns a result set of a single data type; however, it is possible that there will be cases where we need our function to return a result set of multiple fields. For example, let's consider the same function as before, but this time, we want the pk, title pair to be returned as a result, so our function becomes the following:

```
forumdb=> create or replace function delete_posts_table (p_title text)
returns table (ret_key integer,ret_title text) AS $$
delete from posts where title=p_title returning pk,title;
$$
```

```
language SQL;
CREATE FUNCTION
```

The only difference between this and the previous function is that now the function returns a table type; inside the table type, we have to specify the name and the type of the fields. As we have seen before, this is the situation before calling the function:

```
forumdb=> select pk,title from posts order by pk;
pk |      title
----+-----
  5 | Indexing PostgreSQL
  6 | Indexing Mysql
(2 rows)
```

Let's now insert a new record:

```
forumdb=> insert into posts(title,author,category) values ('My new
post',1,1);
INSERT 0 1
```

Now let's call the `delete_posts_table` function. The correct way to call the function is:

```
forumdb=> select * from delete_posts_table('My new post');
ret_key | ret_title
-----+-----
      9 | My new post
(1 row)
)
```

This is the situation after calling the function:

```
forumdb=> select pk,title from posts order by pk;
pk |      title
----+-----
  5 | Indexing PostgreSQL
  6 | Indexing Mysql
(2 rows)
```

The functions that return a table can be treated as real tables, in the sense that we can use them with the `in`, `exists`, `join`, and so on options.

Polymorphic SQL functions

In this section, we will briefly talk about polymorphic SQL functions.

Polymorphic functions are useful for DBAs when we need to write a function that has to work with different types of data. To better understand polymorphic functions, let's start with an example. Suppose we want to recreate something that looks like the Oracle NVL function – in other words, we want to create a function that accepts two parameters and replaces the first parameter with the second one if the first parameter is NULL. The problem is that we want to write a single function that is valid for all types of data (integer, real, text, and so on).

The following function makes this possible:

```
forumbd=> create or replace function nvl ( anyelement,anyelement) returns
anyelement as $$
select coalesce($1,$2);
$$
language SQL;
CREATE FUNCTION
```

This is how to call it:

```
forumbd=> select nvl(NULL::int,1);
nvl
-----
1
(1 row)

forumbd=> select nvl(' '::text,'n '::text);
nvl
-----

(1 row)

forumbd=> select nvl('a '::text,'n '::text);
nvl
-----
a
(1 row)
```

For further information, see the official documentation at <https://www.postgresql.org/docs/current/extend-type-system.html>.

PL/pgSQL functions

In this section, we'll talk about the PL/pgSQL language. The PL/pgSQL language is the default built-in procedural language for PostgreSQL. As described in the official documentation, the design goals with PL/pgSQL were to create a loadable procedural language that can do the following:

- Can be used to create functions and trigger procedures (we'll talk about triggers in the next chapter).
- Add new control structures.
- Add new data types to the SQL language.

It is very similar to Oracle PL/SQL and supports the following:

- Variable declarations
- Expressions
- Control structures as conditional structures or loop structures
- Cursors

First overview

As we saw at the beginning of the *SQL functions* section, the prototype for writing functions in PostgreSQL is as follows:

```
CREATE FUNCTION function_name(p1 type, p2 type, p3 type, ....., pn type)
    RETURNS type AS
BEGIN
    -- function logic
END;
LANGUAGE language_name
```

Now, suppose that we want to recreate the `my_sum` function using the PL/pgSQL language:

```
forumdb=> CREATE OR REPLACE FUNCTION my_sum(x integer, y integer) RETURNS
integer AS
$BODY$
DECLARE
    ret integer;
BEGIN
```

```

    ret := x + y;
    return ret;
END;
$BODY$
language 'plpgsql';
CREATE FUNCTION
forumdb=> select my_sum(2,3);
    my_sum
-----
         5
(1 row)

```

The preceding query provides the same results as the query seen at the beginning of the chapter. Now, let's examine it in more detail:

1. The following is the function header; here, you define the name of the function, the input parameters, and the return value:

```

CREATE OR REPLACE FUNCTION my_sum(x integer, y integer) RETURNS
integer AS

```

2. The following is a label indicating the beginning of the code. We can put any string in between the \$\$ characters; the important thing is that the same label is present at the end of the function:

```

$BODY$

```

3. In the following section, we can define our variables; it is important that each declaration or statement ends with a semicolon:

```

DECLARE
    ret integer;

```

4. With the BEGIN statement, we tell PostgreSQL that we want to start to write our logic:

```

BEGIN
    ret := x + y;
    return ret;

```



Caution: Do not write a semicolon after BEGIN – it's not correct and it will generate a syntax error.

5. Between the BEGIN statement and the END statement, we can put our own code:

```
END;
```

6. The END instruction indicates that our code has ended:

```
$BODY$
```

7. This label closes the first label and at last, the language statement specifies PostgreSQL, in which the function is written:

```
language 'plpgsql';
```

Dropping functions

To drop a function, we have to execute the DROP FUNCTION command followed by the name of the function and its parameters. For example, to drop the my_sum function, we have to execute:

```
forumdb=> DROP FUNCTION my_sum(integer, integer);  
DROP FUNCTION
```

Declaring function parameters

After learning about how to write a simple PL/pgSQL function, let's go into a little more detail about the single aspects seen in the preceding section. Let's start with the declaration of the parameters. In the next two examples, we'll see how to define, in two different ways, the my_sum function that we have seen before.

The first example is as follows:

```
forumdb=> CREATE OR REPLACE FUNCTION my_sum(integer, integer) RETURNS  
integer AS  
$BODY$  
DECLARE  
  x alias for $1;  
  y alias for $2;
```

```
    ret integer;
BEGIN
    ret := x + y;
    return ret;
END;
$BODY$
language 'plpgsql';
CREATE FUNCTION
```

The second example is as follows:

```
forumdb=> CREATE OR REPLACE FUNCTION my_sum(integer, integer) RETURNS
integer AS
$BODY$
DECLARE
    ret integer;
BEGIN
    ret := $1 + $2;
    return ret;
END;
$BODY$
language 'plpgsql';
CREATE FUNCTION
```

In the first example, we used alias; the syntax of alias is, in general, the following:

```
newname ALIAS FOR oldname;
```

In our specific case, we used the positional variable \$1 as the oldname value. In the second example, we used the positional approach exactly as we did in the case of SQL functions.

IN/OUT parameters

In the preceding example, we used the RETURNS clause in the first row of the function definition; however, there is another way to reach the same goal. In PL/pgSQL, we can define all parameters as input parameters, output parameters, or input/output parameters. For example, say we write the following:

```
forumdb=> CREATE OR REPLACE FUNCTION my_sum_3_params(IN x integer, IN y
integer, OUT z integer) AS
$BODY$
```

```

BEGIN
  z := x+y;
END;
$BODY$
language 'plpgsql';
CREATE FUNCTION

```

We have defined a new function called `my_sum_3_params`, which accepts two input parameters (`x` and `y`) and has an output of parameter `z`. As there are two input parameters, the function will be called with only two parameters, exactly as in the last function:

```

forumbd=> select my_sum_3_params(2,3);
my_sum_3_params
-----
                5
(1 row)

```

With this kind of parameter definition, we can have functions that have multiple variables as a result. For example, if we want a function that, given two integer values, computes their sum and their product, we can write something like this:

```

forumbd=> CREATE OR REPLACE FUNCTION my_sum_mul(IN x integer,IN y
integer,OUT w integer, OUT z integer) AS
$BODY$
BEGIN
  z := x+y;
  w := x*y;
END;
$BODY$
language 'plpgsql';
CREATE FUNCTION

```

The strange thing is that if we invoke the function as we did before, we will have the following result:

```

forumbd=> select my_sum_mul(2,3);
my_sum_mul
-----
(6,5)
(1 row)

```

This result seems to be a little bit strange because the result is not a scalar value but a record, which is a custom type. To cause the output to be separated as columns, we have to use the following syntax:

```
forumdb=> select * from my_sum_mul(2,3);
 w | z
---+---
 6 | 5
(1 row)
```

We can use the result of the function exactly as if it were a result of a table and write, for example, the following:

```
forumdb=> select * from my_sum_mul(2,3) where w=6;
 w | z
---+---
 6 | 5
(1 row)
```

We can define the parameters as follows:

- IN: Input parameters (if omitted, this is the default option)
- OUT: Output parameters
- INOUT: Input/output parameters

Function volatility categories

In PostgreSQL, each function can be defined as `VOLATILE`, `STABLE`, or `IMMUTABLE`. If we do not specify anything, the default value is `VOLATILE`. The difference between these three possible definitions is well described in the official documentation (<https://www.postgresql.org/docs/current/xfunc-volatility.html>):



A **VOLATILE** function can do everything, including modifying the database. It can return different results on successive calls with the same arguments. The optimizer makes no assumptions about the behavior of such functions. A query using a volatile function will re-evaluate the function at every row where its value is needed. If a function is marked as **VOLATILE**, it can return different results if we call it multiple times using the same input parameters.

A **STABLE** function cannot modify the database and is guaranteed to return the same results given the same arguments for all rows within a single statement. This category allows the optimizer to optimize multiple calls of the function to a single call. In particular, it is safe to use an expression containing such a function in an index scan condition. If a function is marked as **STABLE**, the function will return the same result given the same parameters within the same transaction.

An **IMMUTABLE** function cannot modify the database and is guaranteed to return the same results given the same arguments forever. This category allows the optimizer to pre-evaluate the function when a query calls it with constant arguments.

In the following pages of this chapter, we will only be focusing on examples of volatile functions; however, here we will briefly look at one example of a stable function and one example of an immutable function:

1. Let's start with a stable function – for example, the `now()` function is a stable function. The `now()` function returns the current date and time that we have at the beginning of the transaction, as we can see here:

```
forumdb=> begin ;
BEGIN

forumdb=*> select now();
              now
-----
```